

PENGUJIAN DAN KEAMANAN API WISATAKU

Nama : Ahmad Damanhuri
NIM : SI21240001
Program Studi : Sistem Informasi
Mata Kuliah : Teknologi Web Service

1. TUJUAN

Pengujian pada API WisataKu dilakukan untuk memastikan bahwa sistem dapat menerapkan mekanisme keamanan dan dapat menjalankan endpoint sesuai dengan hak akses pengguna.

Pengujian mencakup autentikasi menggunakan JSON Web Token (JWT), pengamanan password menggunakan hashing, otorisasi berdasarkan role pengguna atau Role-Based Access Control (RBAC), pengujian endpoint menggunakan Swagger, pengujian otomatis menggunakan Jest dan Supertest, serta pengujian beban menggunakan k6.

Tujuan pengujian ini adalah:

1. Memastikan pengguna dapat melakukan registrasi dan login.
2. Memastikan password pengguna tidak dikembalikan secara langsung dalam response API.
3. Memastikan login menghasilkan JWT yang dapat digunakan untuk mengakses endpoint yang dilindungi.
4. Memastikan endpoint yang dilindungi menolak request tanpa token atau menggunakan token yang tidak valid.
5. Memastikan pengguna dengan role wisatawan tidak dapat mengakses endpoint khusus admin.
6. Memastikan pengguna dengan role admin dapat melakukan operasi penambahan, perubahan, dan penghapusan data destinasi.
7. Memastikan source code dapat melewati Unit Testing dan End-to-End Testing.
8. Mengetahui kemampuan API dalam menangani sejumlah request secara bersamaan menggunakan load testing.

2. IMPLEMENTASI KEAMANAN API

2.1 JWT Authentication

Sistem WisataKu menggunakan JSON Web Token (JWT) sebagai mekanisme autentikasi.

Proses autentikasi dilakukan melalui endpoint:

POST /auth/register
POST /auth/login

Pada saat login berhasil, sistem menghasilkan `access_token` berupa JWT. Token tersebut digunakan pada request berikutnya dengan mekanisme Bearer Token.

Payload JWT yang digunakan membawa informasi pengguna berupa:

sub
email
role
iat
exp

Keterangan:

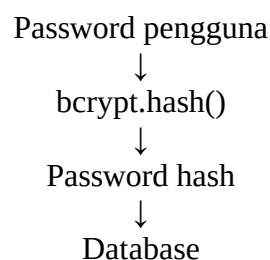
- `sub` merupakan ID pengguna.
- `email` merupakan email pengguna.
- `role` merupakan role pengguna.
- `iat` merupakan waktu token dibuat.
- `exp` merupakan waktu token kedaluwarsa.

JWT digunakan oleh `JwtStrategy` untuk melakukan validasi token sebelum pengguna dapat mengakses endpoint yang dilindungi.

2.2 Password Hashing

Password pengguna tidak disimpan dalam bentuk password asli. Password diproses menggunakan metode hashing dengan library `bcrypt`.

Alur penyimpanan password adalah:



Ketika pengguna melakukan login, password yang dimasukkan akan dibandingkan dengan password hash yang tersimpan menggunakan `bcrypt.compare()`.

Dengan mekanisme tersebut, password asli pengguna tidak perlu disimpan maupun dikembalikan dalam response API.

2.3 JWT Guard

Untuk melindungi endpoint yang membutuhkan autentikasi digunakan JwtAuthGuard.

Guard membaca token dari header:

Authorization: Bearer <token>

Token kemudian diverifikasi menggunakan secret JWT. Jika token valid, data pengguna diteruskan ke request sehingga dapat digunakan dalam proses authorization.

2.4 Role-Based Access Control (RBAC)

Sistem menggunakan Role-Based Access Control untuk membatasi endpoint berdasarkan role pengguna.

Role yang digunakan dalam pengujian adalah:

admin

wisatawan

Endpoint yang diamankan adalah:

Endpoint	Hak Akses
GET /destinasi	Public
GET /destinasi/:id	Public
GET /destinasi/:id/ulasan	Public
POST /destinasi	Admin
PATCH /destinasi/:id	Admin
DELETE /destinasi/:id	Admin

Pembatasan role diterapkan menggunakan decorator **@Roles('admin')** dan **RolesGuard**.

3. PENGUJIAN API MENGGUNAKAN SWAGGER

Pengujian endpoint dilakukan menggunakan Swagger pada:

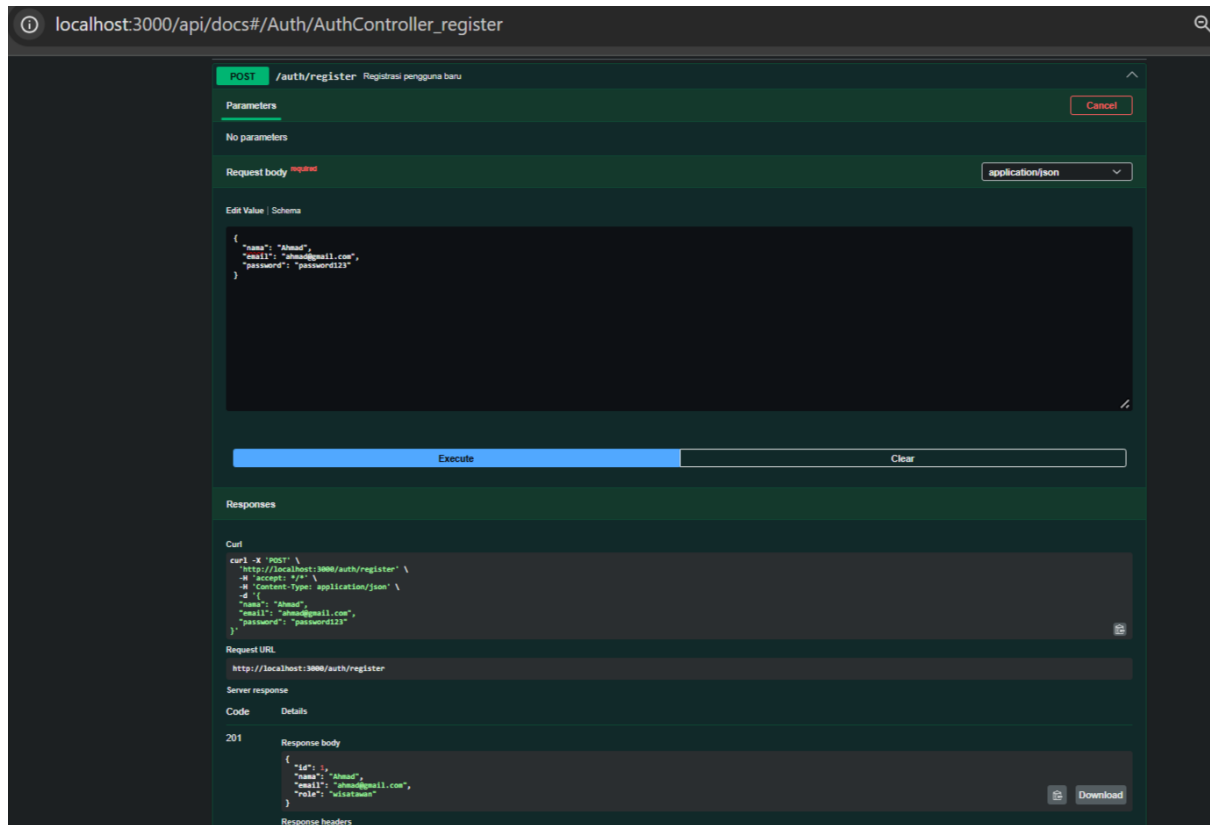
http://localhost:3000/api/docs

Swagger digunakan untuk menjalankan endpoint secara langsung dan mengamati HTTP status code serta response body.

3.1 Pengujian Registrasi Berhasil

Endpoint:

POST /auth/register



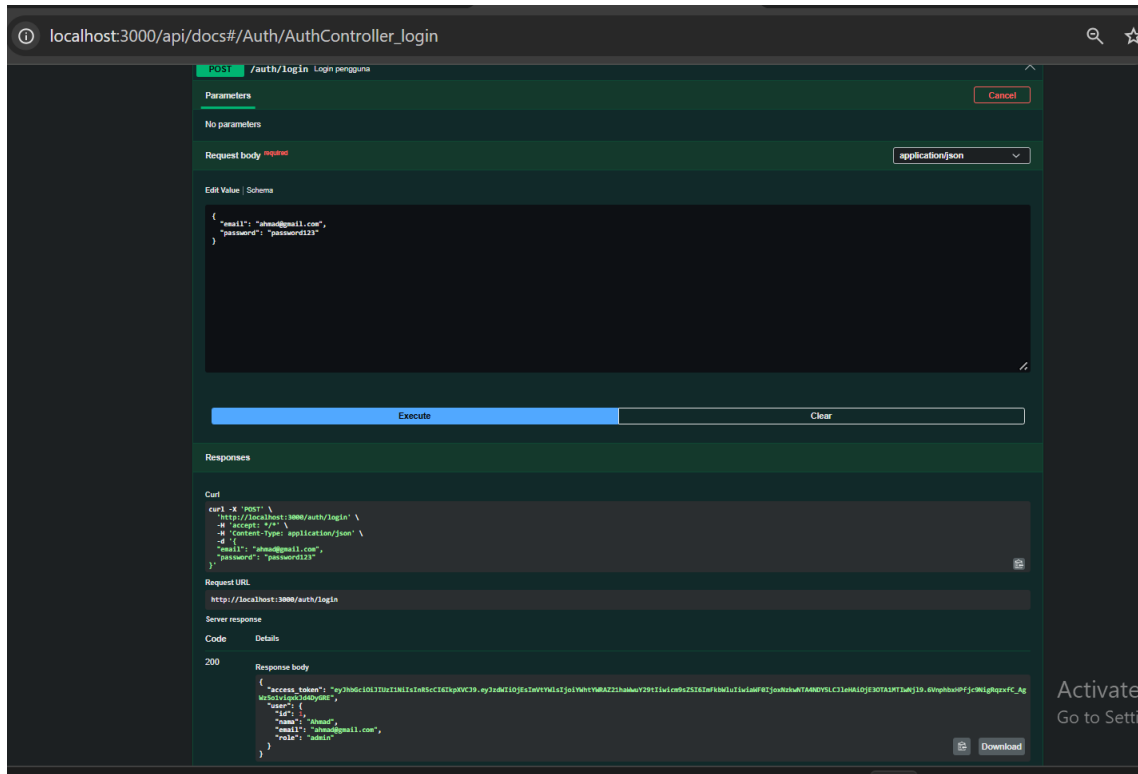
Hasil: Pengujian berhasil.

Password tidak ditampilkan pada response.

3.2 Pengujian Login Berhasil

Endpoint:

POST /auth/login



Response menghasilkan access_token JWT dan informasi pengguna.

Pada pengujian akhir, pengguna memiliki role:

admin

Response memuat:

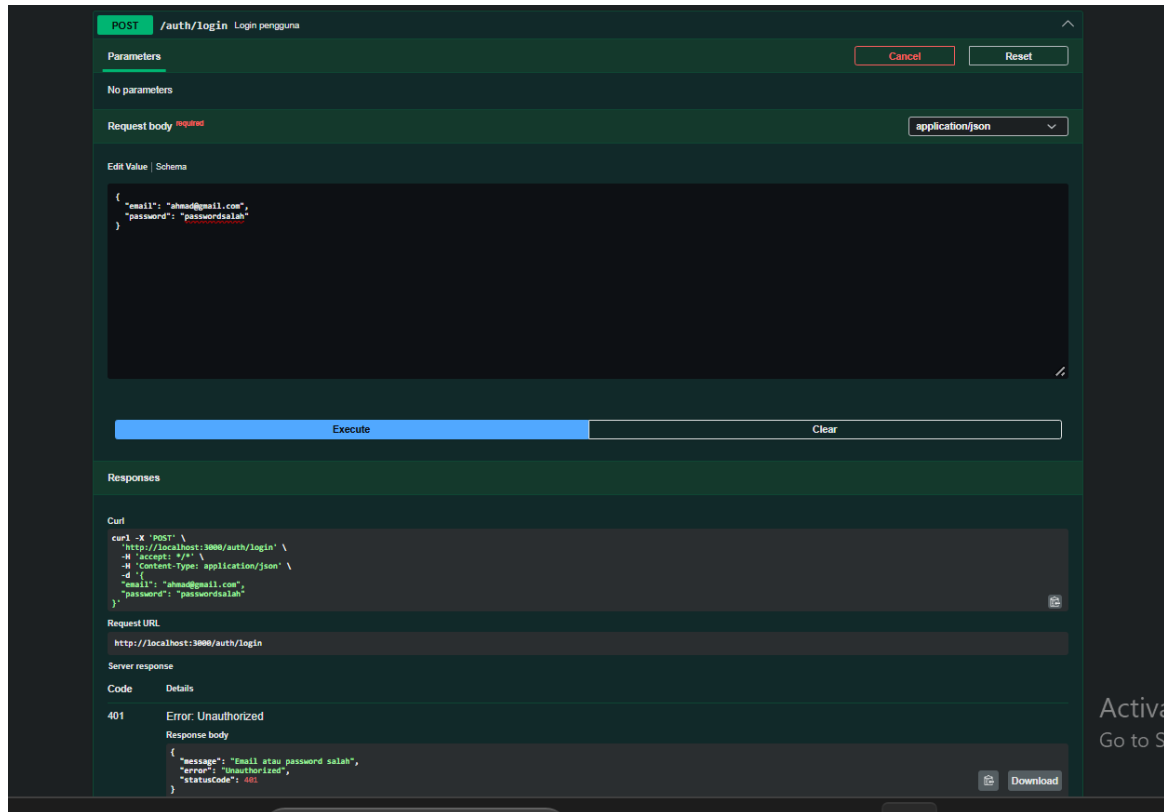
```
{
  "access_token": "JWT_TOKEN",
  "user": {
    "id": 1,
    "nama": "Ahmad",
    "email": "ahmad@gmail.com",
    "role": "admin"
  }
}
```

Hasil: Login berhasil dan JWT berhasil diterbitkan.

3.3 Pengujian Login dengan Password Salah

Endpoint:

POST /auth/login



Hasil: Sistem berhasil menolak login dengan password yang salah.

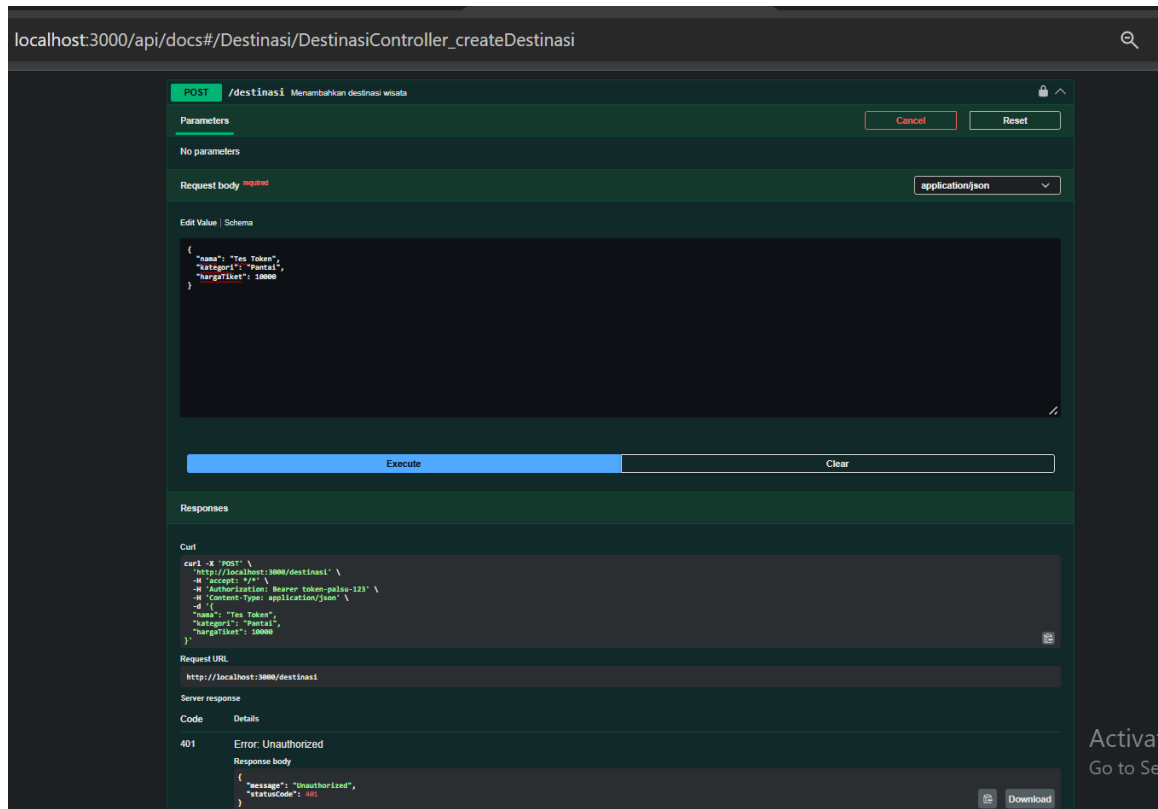
4. PENGUJIAN AUTENTIKASI DAN OTORISASI

4.1 Akses Endpoint Tanpa JWT

Endpoint yang diuji:

POST /destinasi

Request dilakukan tanpa mengirimkan JWT.



Hasil:

HTTP Status: 401 Unauthorized

Hal ini menunjukkan bahwa endpoint tidak dapat diakses tanpa autentikasi.

Response:

```
{
  "message": "Unauthorized",
  "statusCode": 401
}
```

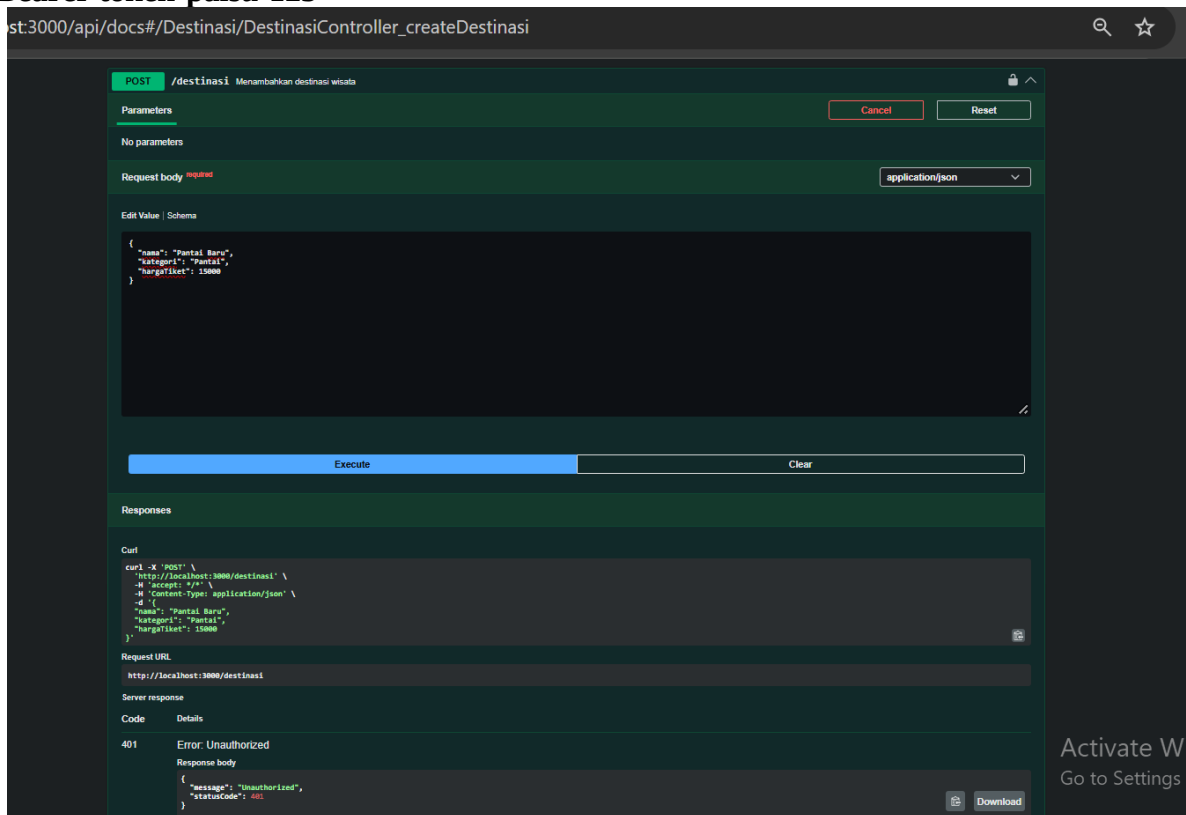
4.2 Akses dengan JWT Tidak Valid

Endpoint:

POST /destinasi

Request menggunakan token palsu:

Bearer token-palsu-123



Hasil:

HTTP Status: 401 Unauthorized

Response:

```
{  "message": "Unauthorized",  "statusCode": 401}
```

Hasil: Sistem berhasil menolak JWT yang tidak valid.

4.3 Role Wisatawan Mengakses Endpoint Admin

Pengguna dengan role:

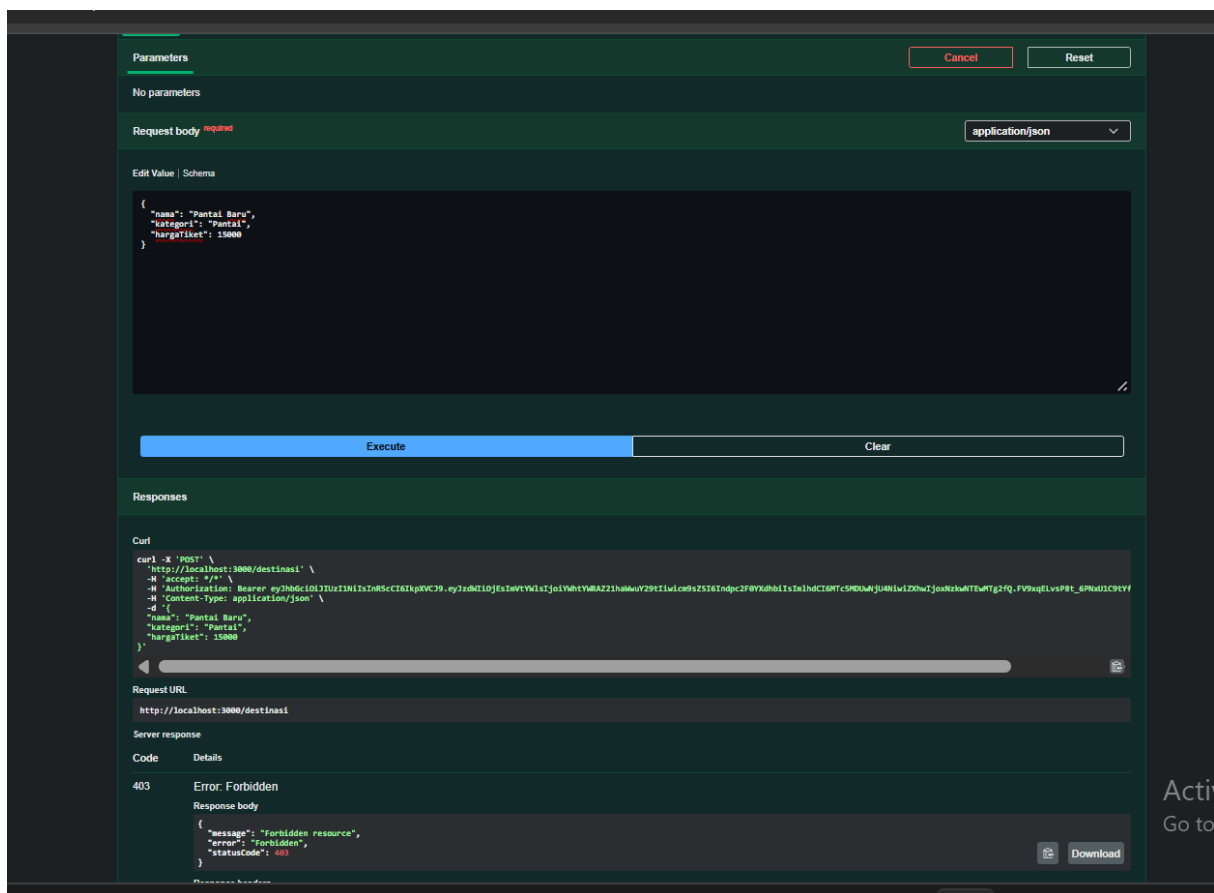
wisatawan

mencoba mengakses:

POST /destinasi

Endpoint tersebut hanya diperbolehkan untuk role admin.

Status 403 menunjukkan bahwa pengguna telah dikenali, tetapi tidak memiliki role yang diperlukan untuk endpoint tersebut.



Hasil:

HTTP Status: 403 Forbidden

Response:

```
{
  "message": "Forbidden resource",
  "error": "Forbidden",
  "statusCode": 403
}
```

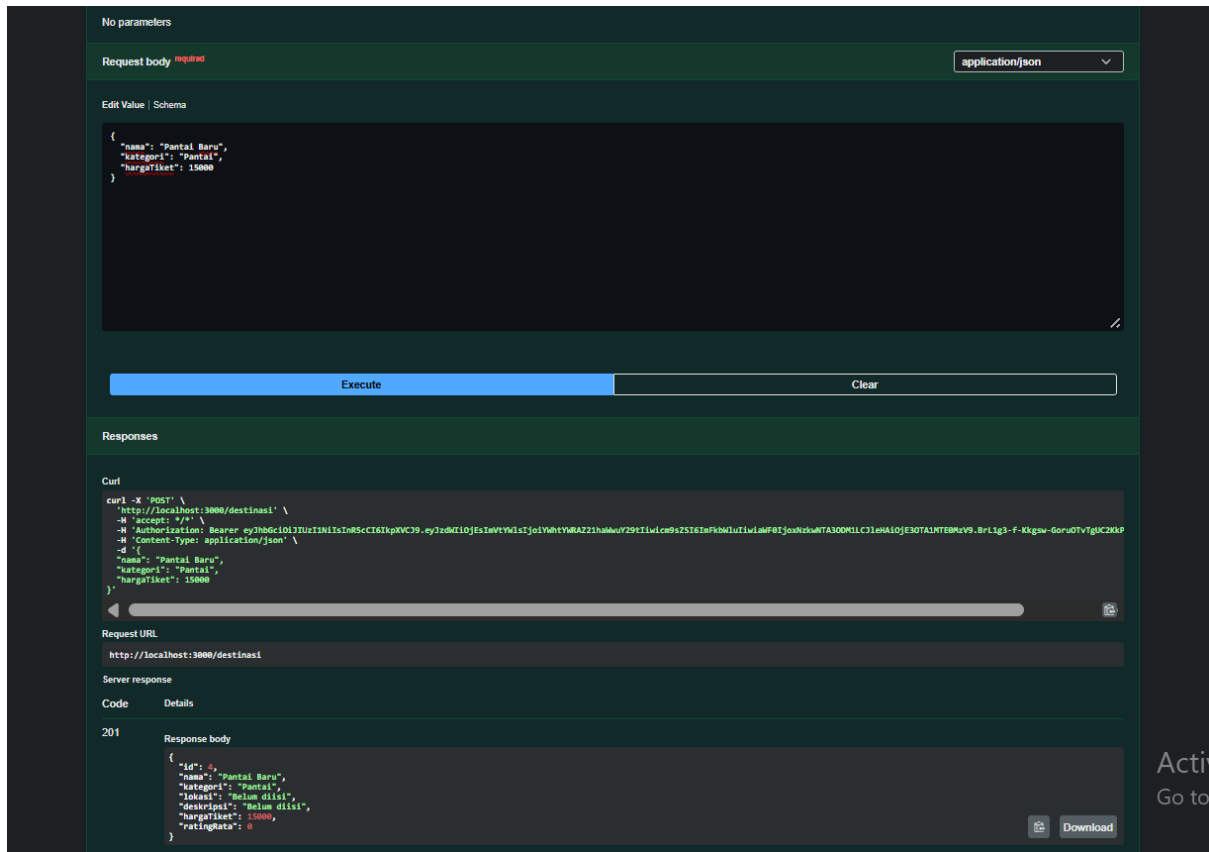
4.4 Role Admin Menambahkan Destinasi

Pengguna dengan role:

admin

mengakses:

POST /destinasi



Data yang digunakan:

```
{
  "nama": "Pantai Baru",
  "kategori": "Pantai",
  "hargaTiket": 15000
}
```

Hasil:

HTTP Status: 201 Created

Response:

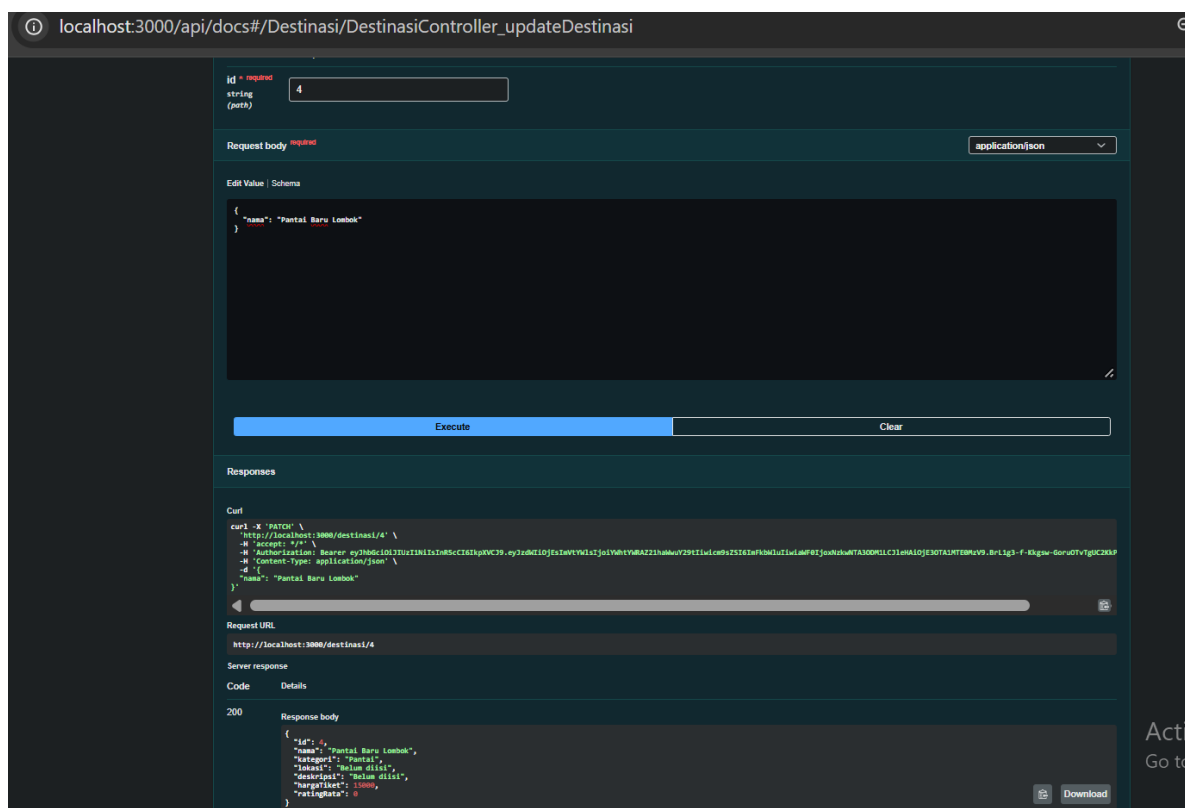
```
{
  "id": 4,
  "nama": "Pantai Baru",
  "kategori": "Pantai",
  "lokasi": "Belum diisi",
  "deskripsi": "Belum diisi",
  "hargaTiket": 15000,
  "ratingRata": 0
}
```

Hasil: Admin berhasil melakukan penambahan data destinasi.

4.5 Role Admin Memperbarui Destinasi

Endpoint:

PATCH /destinasi/4



Data:

```
{
  "nama": "Pantai Baru Lombok"
}
```

Hasil:

HTTP Status: 200 OK

Response:

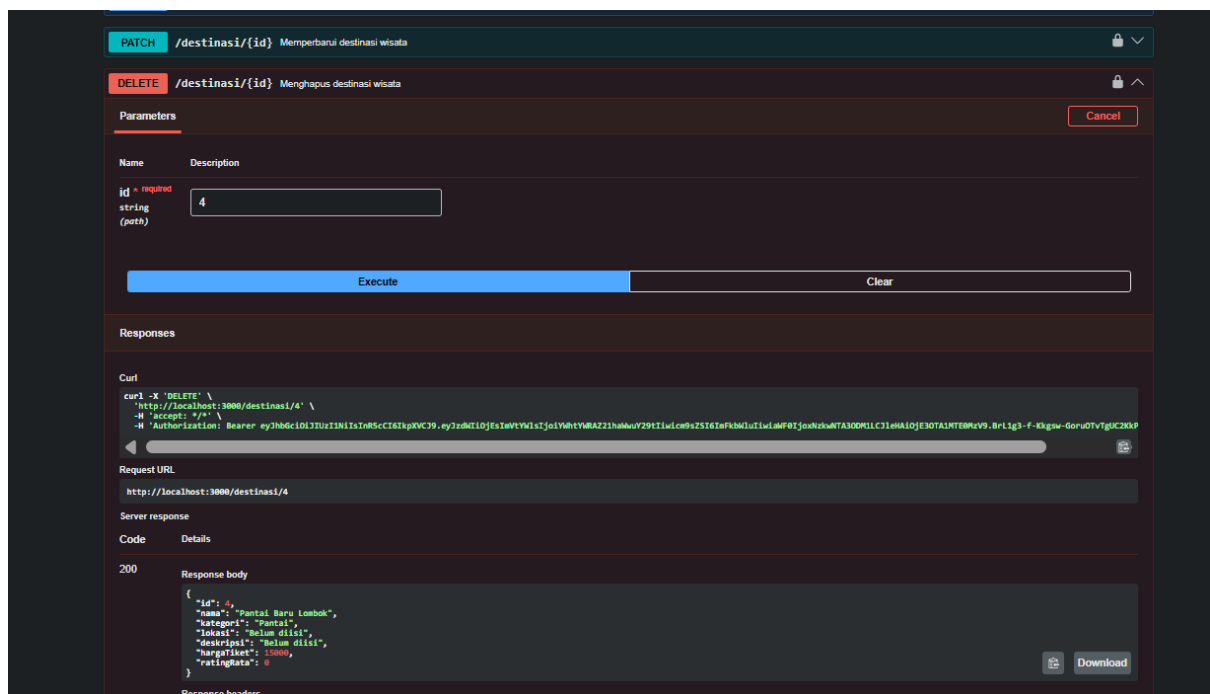
```
{
  "id": 4,
  "nama": "Pantai Baru Lombok",
  "kategori": "Pantai",
  "lokasi": "Belum diisi",
  "deskripsi": "Belum diisi",
  "hargaTiket": 15000,
  "ratingRata": 0
}
```

Hasil: Admin berhasil memperbarui data destinasi.

4.6 Role Admin Menghapus Destinasi

Endpoint:

DELETE /destinasi/4



Hasil:

HTTP Status: 200 OK

Response:

 $\{$

```

    "id": 4,
    "nama": "Pantai Baru Lombok",
    "kategori": "Pantai",
    "lokasi": "Belum diisi",
    "deskripsi": "Belum diisi",
    "hargaTiket": 15000,
    "ratingRata": 0
  }
}

```

Hasil: Admin berhasil menghapus data destinasi.

5. UNIT TESTING MENGGUNAKAN JEST

Unit testing dilakukan untuk menguji komponen atau logic tertentu secara terpisah.

Unit test tambahan dibuat untuk AuthService.

Skenario yang diuji:

1. Registrasi pengguna baru.
2. Penolakan email yang sudah terdaftar.
3. Login berhasil dan penerbitan JWT.
4. Penolakan password yang salah.

```

src > destinasi > TS destinasi.controller.spec.ts > ...
  9  describe('DestinasiController', () => {
    50
    51    it('should be defined', () => {
    52      expect(controller).toBeDefined();
    53    });
    54  });

```

Problems Output Debug Console Terminal Ports

Snapshots: 0 total
Time: 2.651 s, estimated 3 s
Ran all test suites.
PS C:\Users\LENOVO\Documents\Teknologi Web Service\Tugas 2 Desain Endpoint API\wisataku-api> npm.cmd test

```

> wisataku-api@0.0.1 test
> node --experimental-vm-modules ./node_modules/jest/bin/jest.js

(node:7432) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use 'node --trace-warnings ...' to show where the warning was created)
PASS src/app.controller.spec.ts
(node:6664) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use 'node --trace-warnings ...' to show where the warning was created)
PASS src/prisma/prisma.service.spec.ts
(node:17084) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use 'node --trace-warnings ...' to show where the warning was created)
PASS src/auth/auth.service.spec.ts
(node:17752) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use 'node --trace-warnings ...' to show where the warning was created)
PASS src/destinasi/destinasi.controller.spec.ts

```

Test Suites: 4 passed, 4 total
Tests: 7 passed, 7 total
Snapshots: 0 total
Time: 2.78 s
Ran all test suites.
PS C:\Users\LENOVO\Documents\Teknologi Web Service\Tugas 2 Desain Endpoint API\wisataku-api>

Hasil pengujian:

Test Suites: 1 passed, 1 total

Tests: 4 passed, 4 total

Selain itu dilakukan pengujian seluruh Unit Test project.

Hasil akhir:

Test Suites: 4 passed, 4 total

Tests: 7 passed, 7 total

Snapshots: 0 total

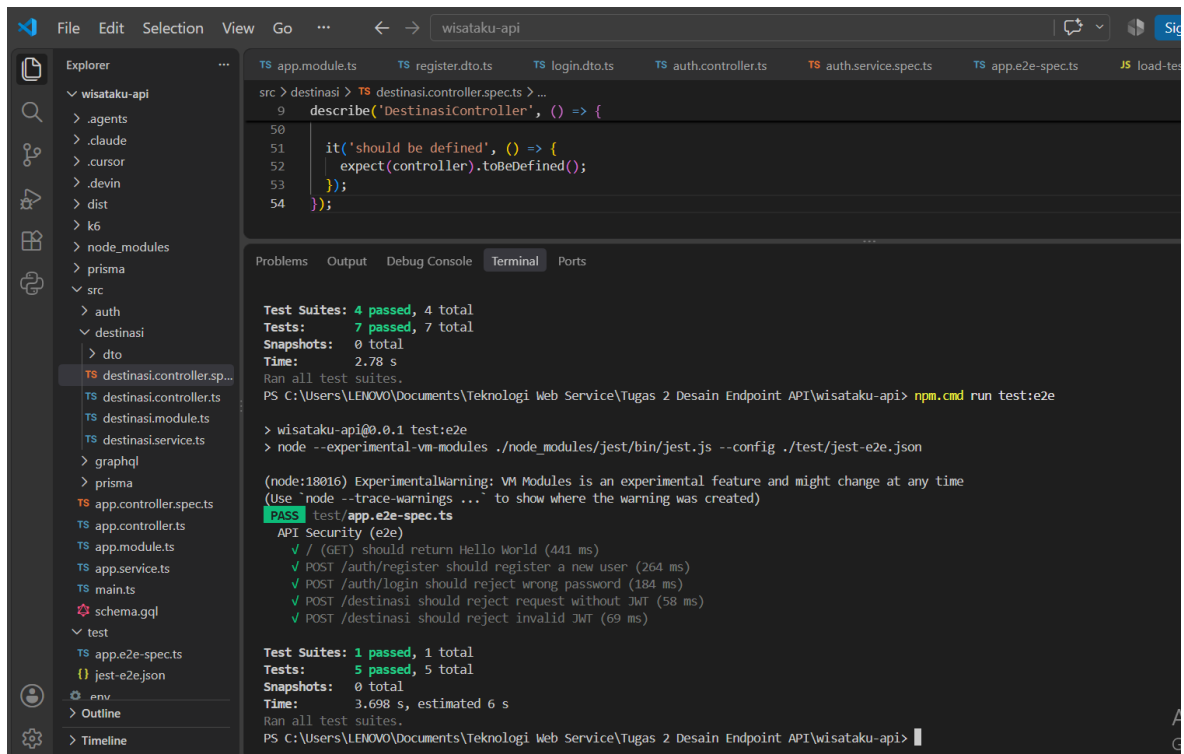
Dengan demikian seluruh Unit Test yang dijalankan berhasil.

6. END-TO-END TESTING MENGGUNAKAN SUPERTEST

End-to-End Testing dilakukan untuk menguji API melalui endpoint secara langsung.

Skenario yang diuji:

1. GET / berhasil.
2. POST /auth/register berhasil.
3. POST /auth/login dengan password salah ditolak.
4. POST /destinasi tanpa JWT ditolak.
5. POST /destinasi dengan JWT tidak valid ditolak.



Hasil akhir:

Test Suites: 1 passed, 1 total

Tests: 5 passed, 5 total

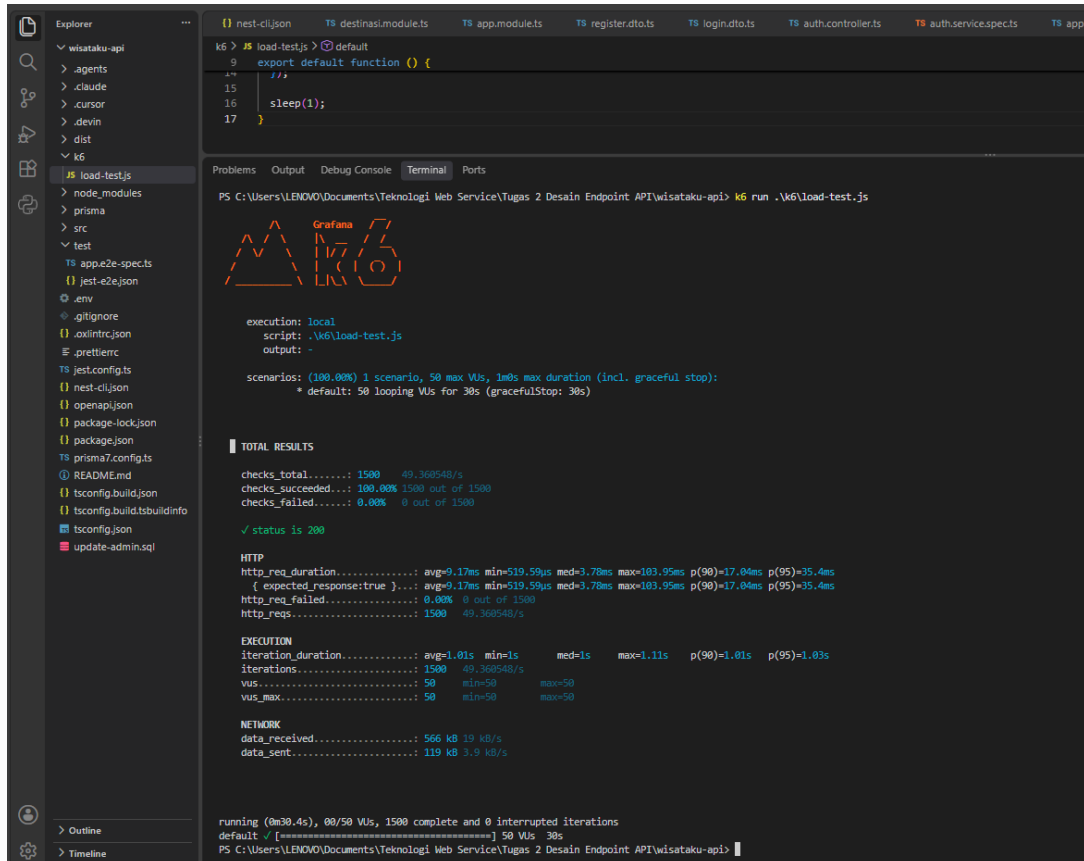
Snapshots: 0 total

Seluruh skenario E2E berhasil.

7. LOAD TESTING MENGGUNAKAN K6

Load testing dilakukan menggunakan k6 terhadap endpoint:

GET /destinasi



The screenshot shows a VS Code editor with a file explorer on the left containing a project named 'wisataku-api'. The main editor displays a k6 script file named 'load-test.js' with the following content:

```
k6 > JS load-test.js > default
9 export default function () {
15 //
16   sleep(1);
17 }
```

The terminal at the bottom shows the command 'k6 run .\k6\load-test.js' being executed. The output includes a summary of the test results:

```
PS C:\Users\LENOVO\Documents\Teknologi Web Service\Tugas 2 Desain Endpoint API\wisataku-api> k6 run .\k6\load-test.js

execution: local
script: .\k6\load-test.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
 * default: 50 looping VUs for 30s (gracefulStop: 30s)

TOTAL RESULTS
checks_total.....: 1500 49.369548/s
checks_succeeded...: 100.00% 1500 out of 1500
checks_failed.....: 0.00% 0 out of 1500
✓ status is 200

HTTP
http_req_duration.....: avg=3.17ms min=519.59µs med=3.78ms max=103.95ms p(90)=17.04ms p(95)=35.4ms
{ expected_response:true }...: avg=3.17ms min=519.59µs med=3.78ms max=103.95ms p(90)=17.04ms p(95)=35.4ms
http_req_failed.....: 0.00% 0 out of 1500
http_reqs.....: 1500 49.369548/s

EXECUTION
iteration_duration.....: avg=1.01s min=1s med=1s max=1.11s p(90)=1.01s p(95)=1.03s
iterations.....: 1500 49.369548/s
vus.....: 50 min=50 max=50
vus_max.....: 50 min=50 max=50

NETWORK
data_received.....: 566 kB 19 kB/s
data_sent.....: 119 kB 3.9 kB/s

running (0m30.4s), 00/50 VUs, 1500 complete and 0 interrupted iterations
default ✓ [=====] 50 VUs 30s
PS C:\Users\LENOVO\Documents\Teknologi Web Service\Tugas 2 Desain Endpoint API\wisataku-api>
```

Konfigurasi pengujian:

Virtual Users : 50 VUs

Durasi : 30 detik

Hasil pengujian:

Metrik	Hasil
Virtual Users	50
Durasi	30 detik
Total Request	1.500
Request Rate	49,36 request/detik
Check Berhasil	1.500 / 1.500
Check Berhasil	100%
Request Gagal	0 / 1.500
Error Rate	0,00%
Average Response Time	9,17 ms

Median	3,78 ms
p90	17,04 ms
p95	35,4 ms
Maximum Response Time	103,95 ms

Hasil k6 menunjukkan bahwa selama skenario pengujian tersebut, seluruh request yang diuji berhasil memperoleh response yang memenuhi check status is 200.

8. HASIL PENGUJIAN

Berdasarkan pengujian yang telah dilakukan, diperoleh hasil sebagai berikut:

No.	Pengujian	Hasil
1	Register pengguna	Berhasil
2	Login pengguna	Berhasil
3	Login password salah	Berhasil ditolak
4	Akses tanpa JWT	Berhasil ditolak
5	JWT tidak valid	Berhasil ditolak
6	Wisatawan mengakses endpoint admin	Berhasil ditolak
7	Admin menambahkan destinasi	Berhasil
8	Admin memperbarui destinasi	Berhasil
9	Admin menghapus destinasi	Berhasil
10	Unit Test AuthService	4/4 PASS
11	Seluruh Unit Test	7/7 PASS
12	E2E Test	5/5 PASS
13	Load Test k6	1.500 request, 0% gagal
14	Build project	Berhasil

9. KESIMPULAN

Berdasarkan implementasi dan hasil pengujian, API WisataKu telah menerapkan autentikasi menggunakan JWT serta otorisasi berdasarkan role pengguna.

Proses registrasi dan login berhasil dilakukan, sedangkan login dengan password yang salah dapat ditolak menggunakan status 401 Unauthorized. Endpoint yang dilindungi juga berhasil menolak request tanpa JWT maupun menggunakan JWT yang tidak valid.

Implementasi RBAC berhasil membatasi akses berdasarkan role. Pengguna dengan role wisatawan mendapatkan response 403 Forbidden ketika mencoba mengakses endpoint yang membutuhkan role admin, sedangkan pengguna dengan role admin berhasil melakukan operasi POST, PATCH, dan DELETE pada endpoint destinasi.

Pengujian otomatis menggunakan Jest menghasilkan 7 dari 7 test berhasil, sedangkan End-to-End Testing menggunakan Supertest menghasilkan 5 dari 5 test berhasil.

Pengujian beban menggunakan k6 dengan 50 Virtual Users selama 30 detik menghasilkan 1.500 request, dengan 0,00% request gagal, rata-rata waktu response 9,17 ms, dan p95 sebesar 35,4 ms pada skenario pengujian yang dilakukan.

Berdasarkan seluruh hasil tersebut, implementasi keamanan dan pengujian pada API WisataKu dapat dinyatakan telah berjalan sesuai dengan skenario pengujian yang dilakukan.